MongoDB Tutorial

What is Database?

A database is an organized collection of structured information or data, typically stored electronically in a computer system. It allows for efficient storage, retrieval, management, and updating of data, usually managed by a Database Management System (DBMS). Databases are crucial for modern applications, supporting secure, fast access and data integrity.

Types of Databases (SQL vs NoSQL)?

Databases are generally categorized into two main paradigms: SQL (Relational) and NoSQL (Non-relational). Each is designed with different architectural priorities for data structure, scalability, and consistency.

SQL Databases (Relational)?

SQL databases store data in a structured, tabular format using rows and columns. They are ideal for applications where data relationships are complex and data integrity is paramount.

- **Structure:** Uses a predefined schema; data must be organized into fixed tables and columns before entry.
- **Scalability:** Typically, **vertically scalable**, meaning they handle increased load by adding more power (CPU, RAM, SSD) to a single server.
- **Transactions:** Follow strict ACID properties (Atomicity, Consistency, Isolation, Durability) to ensure reliable transaction processing.
- **Querying:** Uses Structured Query Language (SQL), a standardized language for complex joins and data manipulation.
- **Examples:** MYSQL, SQL, PostgreSQL, Oracle Database.

NoSQL Databases (Non-Relational)?

NoSQL ("Not Only SQL") databases store data in flexible formats such as documents, graphs, or key-value pairs. They are optimized for high-volume, unstructured data and rapid development.

- **Structure:** Uses a dynamic schema; data can be stored without a predefined format, allowing for varied attributes within the same collection.
- **Scalability:** Designed for horizontal scalability, allowing they to handle massive traffic by adding more servers (nodes) to a cluster.

- **Consistency:** Often follows the BASE model (Basically Available, Soft state, Eventual consistency), prioritizing availability and speed over immediate consistency.
- **Querying:** Varies by database type; may use JSON-like syntax, APIs, or specialized languages like Cypher for graphs.
- **Examples:** MongoDB, Redis, Cassandra, Neo4j.

What is NoSQL?

MongoDB is an open-source, document-oriented NoSQL database designed to store and manage large volumes of data efficiently using a flexible, JSON-like document model.

- Stores data in BSON format for efficient handling of JSON-like documents.
- Supports dynamic schemas, making it suitable for unstructured or semi-structured data.
- Scalable and easy to learn, widely used in modern web and mobile applications.
- Open-source and freely available on Linux, Windows, and macOS.

Install MongoDB & MongoDB Compass:

- Download MongoDB Community Server:
    - https://www.mongodb.com/try/download/community
- MongoDB Compass:
    - When Install the server there compass already here.
- MongoDB Shell Download:
    - https://www.mongodb.com/try/download/shell

Note: After Installing set environment variables: server and mongosh

JSON VS BSON?

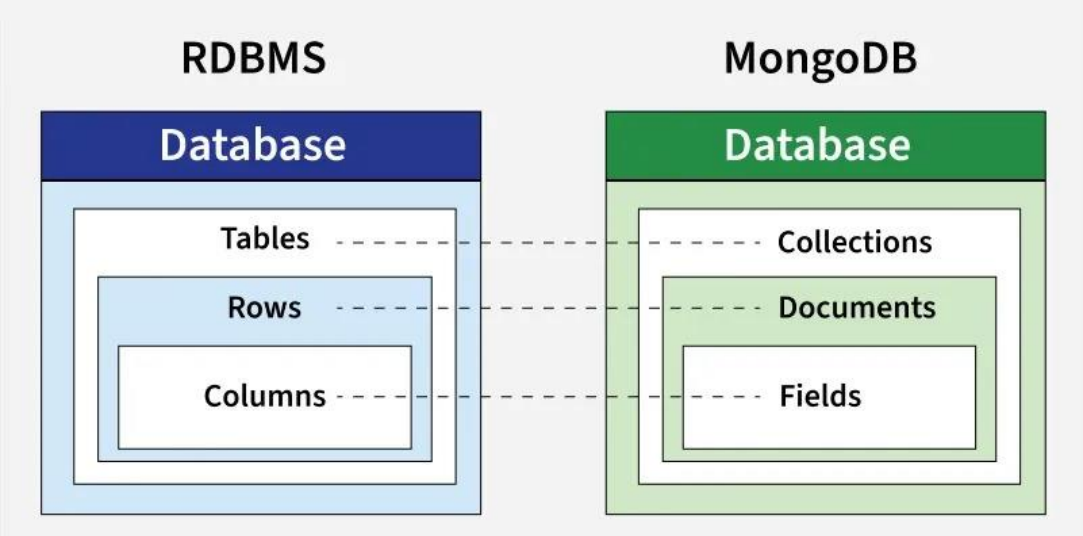
JSON and BSON are data interchange formats built on key-value pairs, but JSON is a human-readable, text-based format for general data exchange, while BSON is a binary-encoded format optimized for efficient data storage and faster machine processing, primarily used by MongoDB.

Feature	JSON	BSON
Format Type	Text-based (human-readable)	Binary-based (machine-readable)
Readability	Yes, easily readable by humans	No, not human-readable
Data Types Supported	Strings, numbers, booleans, arrays, null	All JSON types + Date, Binary, ObjectId, and others
Space Efficiency	Less efficient for complex data types	More efficient due to binary encoding
Performance	Slower parsing and retrieval	Faster parsing and retrieval, especially for large data
Use Case	Data exchange between client and server	Data storage, especially in MongoDB
Compression	No built-in compression	More compact due to binary encoding
Support for Large Data	JSON may struggle with very large data	BSON supports large documents and binary data natively

Advantages of BSON:

- Efficient Storage: BSON's binary format reduces data size compared to JSON, especially when handling large datasets.
- Speed: The binary encoding allows for faster reading and writing of data compared to the textual format of JSON.
- Extended Data Types: BSON supports more advanced data types like Date, Binary, and ObjectId, which are crucial for handling special cases such as timestamps and unique identifiers.

RDBMS VS MongoDB?



Create a Database In MongoDB:

In MongoDB, a database is created only when data is inserted.

use database

Check Existing Databases:

Command: show dbs

This will return the name of the currently selected database

Command: db

Create a Collection in MongoDB:

A collection in MongoDB is a group of schema-less documents, similar to a table in a relational databases, created explicitly or automatically on insertion.

Example: db.createCollection('Students')

InsertOne Method:

Inserting a document automatically creates the collection if it doesn't exist.

db.Students.insertOne({Name: "Farhan", age: 23})

insertMany() Method:

insertMany() method inserts multiple documents in one operation, improving performance over single inserts

Example:

db.Students.insertMany([{name:"Providers", country:"PK"}, {name:"Ali", age:20}])

View Existing Collections

After inserting data, list all collections using the show collections command.

Command: show collections

Drop Existing Collections & Database

Command: db.students.drop()

db.dropDatabase()

InsertOne Method:

Inserting a document automatically creates the collection if it doesn't exist.

db.Students.insertOne({Name: "Farhan", age: 23})

insertMany() Method:

insertMany() method inserts multiple documents in one operation, improving performance over single inserts

Example:

db.Students.insertMany([{name:"Providers", country:"PK"}, {name:"Ali", age:20}])

Read Record:

This command is used for getting all data.
db.users.find()

Read Specific Record:

This command is get only one record.
db.users.find({ age: 25 })

Read Condition Record:

This command is get specific data. Get data using condition.
db.users.find({ age: 25 })

Update Record:

updateOne() method:
db.users.updateOne(
{ name: "Ali" },
{ $set: { age: 26 } }
)

updateMany()

db.users.updateMany(
{ city: "Karachi" },
{ $set: { country: "Pakistan" } }
)

Delete Record:

```
db.users.deleteOne({ name: "Ali" })
```

deleteMany()

```
db.users.deleteMany({ age: { $lt: 25 } })
```